

Soul Reaper: The Hero's Last Stand RPG Game

Project Report

Melisa Söğüt
Computer Engineering
Galatasaray University
23401055

Aslan Yusuf Zeybek
Computer Engineering
Galatasaray University
23401045

I. INTRODUCTION

A. General Overview

"Soul Reaper: The Hero's Last Stand", a console-based, text-driven Object-Oriented Role-Playing Game (RPG) developed in Java as the final course project for the Object-Oriented Programming course.

B. Inspiration: The World of Soul Reapers

The game is inspired by classical supernatural anime lore revolving around Soul Reapers (*Shinigamis*) and the spiritual balance of the universe. In this universe, Shinigamis are divine guardians tasked with purifying corrupted spirits, guiding wandering souls to the afterlife, and protecting human dimensions from spiritual breaches. Combat in this universe is deeply tied to spiritual pressure (*Soul Power*) and specialized weapons called *Zanpakutos* (swords). This project translates these rich thematic elements such as the scaling threat of Hollows evolving into elite Arrancars or Vasto Lordes, and the supportive roles of tactical spiritual boosters into a game.

II. GAME OVERVIEW

A. Core Gameplay Loop

The main gameplay flow is controlled by the `GameEngine` class. The narrative of game centers on a lone playable guardian as we call "Hero" who must navigate through a map divided into hostile zones like Karakura Town, Hueco Mundo and Soul Society. In this game there are 2 map options that stays as like mystery to the Hero unless the regarding items bought for seeing the map. The game operates through a deterministic, turn-based combat loop, first the hero attacks /defends or tries to escape then the enemy attacks. The core objective is to progress through tactical encounters across different regions, manage a localized financial budget to upgrade equipment at the Shop, and survive escalating waves of hostile entities.

The game follows this sequence:

- 1) The game starts from `GameEngine`, which initializes the player, shop, combat system, map state, support characters, and counters. A short text introduction describes the atmosphere to the player. To make the game feel more dramatic and immersive.

- 2) The player selects a difficulty mode which consists of two different mystery maps `GameEngine.createMap(...)` builds one of two six-region enemy layouts.
- 3) The hero moves region by region and encounters enemies such as Hollow, Arrancar, and finally VastoLorde.
- 4) `CombatSystem.startFight(...)` runs a turn-based loop where the hero can attack, defend, use inventory, or attempt escape depending on their luck because escape move has low chance.
- 5) When the Hero defeats a standard enemy, the system checks the enemy's type to calculate rewards. The player instantly wins a specific amount of coins based on the enemy's power and defeated enemies may drop gifts through the `Lootable` contract which are automatically added to the inventory for later use.
- 6) Before the final enemy, two friendly support NPC Shinigami characters apply healing and damage boosting effects to our Hero named after the creators of the game. One Shinigami acts as a Healer to restore the Hero's health, while the other acts as a Booster to increase the damage power of the Hero's active sword. After doing their jobs, these support characters leave the battle zone, leaving the Hero and the Lord face-to-face alone.
- 7) **The Final Boss Battle:** After defeating all standard enemies in the regions, the game moves to the final climax: a fight against the elite VastoLorde, which is the strongest enemy in the game.
- 8) **Game Over Conditions (Winning or Losing):**
 - **Victory:** If the Hero defeats the Vasto Lorde, they win the Lord's legendary sword, receive a massive money reward, and save all the innocent souls to the heaven.
 - **Defeat:** If the Hero's health drops to 0 at any stage of the game, they lose. The Hero's soul is captured by the enemies so the other spirits' souls, and the game ends.
- 9) The run ends with victory, defeat, or replay control. A maximum of two playthroughs is enforced by `OverLoopException`. If required you can restart the game for more than 2 rounds.

- 10) **Saving Data:** Summary metrics like defeated enemies, earned items etc. are automatically written to an external text file. `FileWriter`

B. Player Decisions

During gameplay, the player makes decisions in several areas:

- **Combat choices:** Attack enemies (`attack()`) with a chance of hitting a critical strike. Defend against incoming attacks (`defend()`) to reduce half of the damage. The defense mechanism also includes a probability-based counter-attack feature which allows player to automatically strike back upon successfully defending.
- **Post-Game Statistics:** End-game metrics (enemies defeated, gifts collected, remaining budget) are automatically written to an external text file. Use, equip and view the current inventory (`ArrayList<Item>`). Visit the Shop to buy new equipment within their budget constraints Or try to escape the battle based on a low-percentage luck algorithm.
- **Economic choices:** spend coins in the Shop or save budget for later upgrades.
- **Inventory choices:** equip swords and armours, consume variety of healing potions, use tips for a limited map information or reveal the whole map information with map scroll item.
- **Risk choices:** attempt low-probability escape from normal enemies, while the final boss blocks the escape.

III. DESIGN DECISIONS & JUSTIFICATIONS

This section explains the technical justifications behind our object-oriented design choices regarding of this game .

A. Inheritance and Abstract Class Framework

To prevent code duplication and build a logical hierarchy, we implemented a deep inheritance framework. At the top of our entity design sits the `BaseSoul` abstract class.

- **BaseSoul Part:** Since a generic "Soul" cannot exist in the game without a concrete identity, making `BaseSoul` abstract prevents accidental instantiation. It safely encapsulates shared attributes like `currentHp`, `maxHp`, `Position` etc. ensuring every derived class inherits a consistent state.
- **The Shinigami Branch:** The `Shinigami` extends `BaseSoul` to handle features specific to spiritual guardians. It concretely branches into the playable `Hero`
- **The Final Boss Battle Polymorphic Support:** When the `Hero` player defeats all the enemies before `Lord`, here comes the last part of the game as states. To aid the `Hero` in this challenging battle, two friendly NPC `Shinigamis` arrive as support: *Melisa* and *Aslan*. The system utilizes polymorphism by managing these helper NPCs under a generic `ArrayList<Shinigami>` collection. At runtime, the engine iterates through this list ; one concrete instance acts as a `Healer` to restore the `Hero's` health, while the other acts as a `Booster` to scale the damage

power of the `Hero's` active sword. After executing their unique overridden behaviors. `OverLoopException` and locking the application thread to guarantee stability during debugging routines.

- **The Enemy Branch:** The abstract class `Enemy` extends `BaseSoul` and manages basic hostile logic. It specializes into concrete subclasses: `Hollow`, `Arrancar`, and `VastoLorde`. Each subclass overrides behaviors like `attack()` ,uses `super.methods()` (as required by the project rules) and defines its own dynamic loot drop formulas.
- **The Item Branch:** The abstract class `Item` enforces a uniform structure for all collectibles by declaring an abstract `use(BaseSoul target)` method. This allows subclasses (`Sword`, `Armour`, `HealPotion`, `Tip`, `MapScroll`) to implement their specific features internally without breaking the inventory structure.

B. Interface-Driven Architecture

Interfaces are utilized to enforce behavior contracts across completely different class structures:

- **Attack:** Implemented by both `Hero` and `Enemy`. It separates offensive operations from baseline character stats, allowing the combat loop to process damage calculations polymorphically.
- **Usable:** Implemented by `Item` and `Shinigami`. This contract ensures that any object with "using" capabilities can safely execute actions on a `BaseSoul` target.
- **Lootable:** Implemented by `Enemy` to standardize the reward drop functionality (`drop()`) independently from the baseline enemy combat statistics.
- **Saveable:** Implemented by `GameEngine` to enforce data persistence standards for recording gameplay logs.

C. Concrete Examples of Polymorphism in Source Code

We used polymorphism throughout the core operational loops of our game engine. Here are some of them:

1) *Dynamic Method Overriding (Item Execution & Combat Systems):* The primary execution of polymorphism occurs within the `useItem(Item item)` method inside the `Hero` class: `item.use(this)` Even though the player's inventory holds objects under the generic `Item` superclass, Java dynamically binds the call to the overridden `use()` method of the concrete subclass at runtime. For instance, if the object is a `MapScroll`, it handles map matrix rendering. If it is a `Tip` object, it parses upcoming regions to give the tip etc..

A similar execution loop is utilized in the combat system through the `ArrayList<Shinigami>` and `Enemy[]` collections:

- **Ally Lifecycles:** During the final boss confrontation, support NPCs arrive and are iterated through polymorphically. Even though one concrete instance acts as a `Healer` and another as a `Booster`, the engine drives their behaviors seamlessly via their common supertype.
- **Turn-Based Encounters:** The static `Enemy[]` array manages diverse subclass instances such as `Hollow`,

Arrancar, and VastoLorde. When individual encounter routines execute, program automatically evaluates the concrete identity of each element at runtime to trigger subclass specific combat logic.

2) *Decoupling via Superclass Method Signatures*: Inside the method signatures of `use(BaseSoul target)` in `MapScroll` and `showOneEnemy(BaseSoul target)` in `Tip`, the parameter is passed strictly as the `BaseSoul` parent class. By avoiding a concrete dependency on the `Hero` class, the map and scouting subsystems remain entirely decoupled. This allows them to read position fields from any current or future subclass entity within the game hierarchy.

3) *Safe Downcasting and Copying in Financial Transactions*: In `Shop.sellItem()`, the system manages its catalog via a generic `ArrayList<Item>`. To prevent shallow copying—where modifying player gear would alter the shop’s global stock—the engine uses the `instanceof` operator at runtime to verify the item’s concrete identity. Once identified, the variable is downcast to its specific subclass (e.g., `Sword` or `Tip`), enabling safe access to subclass-specific components to instantiate an independent copy for the player.

D. Exception Handling Strategy

To prevent invalid gameplay states and illegal moves, a multi-layered exception handling strategy is enforced across the system. Instead of allowing the game thread to crash when a player encounters an error state (such as running out of gold or exceeding replay limits), specific operations are wrapped inside localized `try-catch-finally` blocks.

1) *Details of Custom Exceptions*: To prevent invalid game states and illegal moves, we use a multi-layered exception handling strategy throughout the system. As shown in the UML diagram.

- **IOException**: A built-in checked exception caught inside the `save()` method of the `GameEngine` class. It handles infrastructure failures during file system operations when utilizing the `FileWriter`.
- **InsufficientBudgetException**: Thrown when the `Hero` attempts to purchase an item, equipment upgrade etc from the `Shop` without having enough gold in their balance. This guarantees that currency values never drop below zero.
- **InsufficientHealthException**: Thrown when a specific gameplay action requires a minimum health threshold, but the `Hero`’s current HP is too low to safely execute it. This prevents vital actions from being performed in an invalid state.
- **MaxHpExceededException**: Thrown when a healing action or item (such as a `HealPotion` or support from a `Healer` `Shinigami`) attempts to restore the `Hero`’s health beyond their maximum allowable HP (`maxHp`).
- **InvalidSelectionException**: Thrown within console execution menus such as map selection, inventory sorting or shop interfaces if the player enters an invalid index, out-of-bounds choice or faulty string input.

- **CombatAlreadyActiveException**: Thrown if the game engine attempts to initiate a new exploration phase or combat sequence while the player is already locked inside an ongoing battle within the `startFight()` method, protecting the game loop from overlapping states.
- **SwordAlreadyEquippedException** & **ArmourAlreadyEquippedException**: Thrown when the player attempts to equip a weapon or armor piece from their inventory when they already have that exact equipment active in that specific slot.
- **OverLoopException**: Thrown at the very entrance of the game engine lifecycle if the user attempts to trigger a third gameplay loop, enforcing the system’s mandatory maximum replay limitation.

IV. MANDATORY ELEMENTS IN THE SOURCE CODE

In this section, the mandatory elements required for this project are presented. Abstract-concrete classes hierarchy and criterias can be seen in the UML diagram and table given below. All implemented structures fully meet with the all project criterias and guidelines.

TABLE I
MAIN ARCHITECTURAL LAYERS

Layer	Main Classes	Responsibility
Game control	<code>GameEngine</code> , <code>CombatSystem</code>	Controls setup, replay limit, movement, turn order, victory and defeat conditions.
World model	<code>GameMap</code> , <code>Region</code> , <code>Position</code>	Represents the map, region difficulty, enemy placement, and entity locations.
Character model	<code>BaseSoul</code> , <code>Shinigami</code> , <code>Hero</code> , <code>Enemy</code>	Defines shared soul state and separates friendly and hostile entities.
Enemy types	<code>Hollow</code> , <code>Arrancar</code> , <code>VastoLorde</code>	Implements different attacks, rewards, difficulty levels, and loot drops.
Item system	<code>Item</code> , <code>Sword</code> , <code>Armour</code> , <code>HealPotion</code> , <code>Tip</code> , <code>MapScroll</code>	Provides a polymorphic inventory and shop catalog.
Support and utility	<code>Healer</code> , <code>Booster</code> , <code>Shop</code> , <code>InputHelper</code>	Adds support actions, purchasing, input validation, and feedback.

TABLE II
SOURCE CODE COMPONENTS IN GENERAL

Category	Classes / Interfaces
Interfaces	<code>Attack</code> , <code>Usable</code> , <code>Lootable</code> , <code>Saveable</code>
Base and character classes	<code>BaseSoul</code> , <code>Shinigami</code> , <code>Hero</code> , <code>Healer</code> , <code>Booster</code>
Enemies	<code>Enemy</code> , <code>Hollow</code> , <code>Arrancar</code> , <code>VastoLorde</code>
Items	<code>Item</code> , <code>Sword</code> , <code>Armour</code> , <code>HealPotion</code> , <code>Tip</code> , <code>MapScroll</code>
Game systems	<code>GameEngine</code> , <code>GameMap</code> , <code>Region</code> , <code>Position</code> , <code>CombatSystem</code> , <code>Shop</code> , <code>InputHelper</code>
Exceptions	(see Section III-D for details).

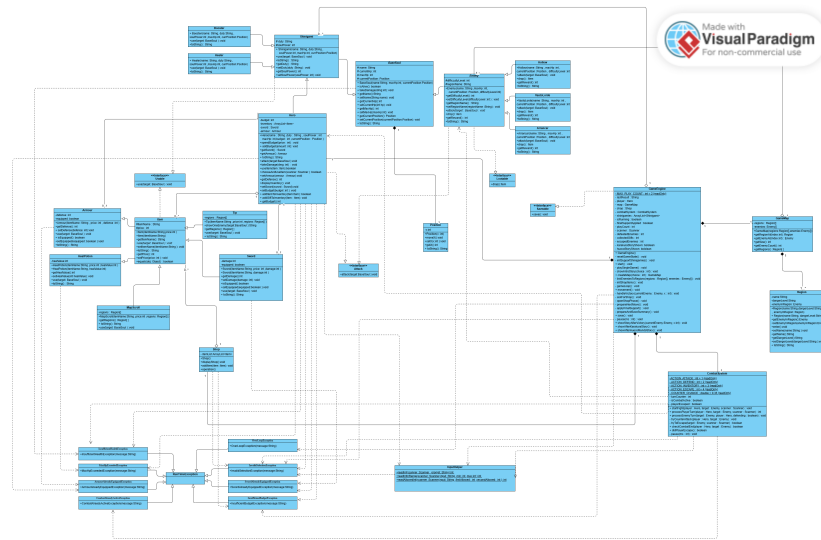


Fig. 1. UML class diagram of the game architecture.

Fig. 2. Mandatory OOP Requirements Table

MANDATORY ELEMENTS	Complete Project Implementation Details
Abstract Classes	BaseSoul, Enemy, Item
Class Hierarchy	This is shown above and fully meets with the criteria. Fully implemented across souls, concrete enemies, and shop items, completely meeting the project criteria.
At least 3 Interfaces	Attack, Usable, Lootable, Saveable.
Abstract Method Overriding	Overridden across multiple concrete classes to handle unique behaviors: toString(), Item drop(), void attack(BaseSoul target), and void use(BaseSoul target) etc.
Polymorphic Collections	Utilized across core loops and arrays: ArrayList<Item> itemList, ArrayList<Shinigami> shinigamis (tracks allies Healer and Booster polymorphically), ArrayList<Item> inventory (holds items under generic supertype), and Enemy[] enemies (stores concrete subclasses like Hollow, Arrancar, and VastoLorde).
toString() & equals()	toString() is implemented in all concrete subclasses (Hollow, Arrancar, Sword, Hero, etc.). The equals() method is overridden and used inside addItemToInventory(Item item) in the Hero class to compare items by attributes (itemName, price) instead of memory addresses, blocking duplicate weapons, armors or items like map scroll.
Instanceof & Casting	Used for runtime identification and downcasting: Shop.sellItem() uses them to discover an item's true concrete type for independent cloning; Sword.use() and Armour.use() check if the target is an instanceof Hero to safely downcast and call hero.setSword(this); CombatSystem.tryToEscape() checks if the target is an instanceof VastoLorde to block escape choices during the final boss fight. And more examples can be found in the source code.
Custom Exception Classes	8 custom exceptions used: CombatAlreadyActiveException, InsufficientBudgetException, InsufficientHealthException, InvalidSelectionException, MaxHpExceededException, OverLoopException, SwordAlreadyEquippedException, and ArmourAlreadyEquippedException.
Try-Catch-Finally Blocks	Used inside the GameEngine, Hero, CombatSystem and InputHelper classes to catch boundary errors, crashes and provide feedback messages.
Super Keyword & Protected	protected modifiers are used for fields in base classes. super() is called in all subclass constructors. The super.method() syntax is explicitly utilized within two key overridden methods: getReward() and toString(). More can be seen in the source code
Playable Game Loop	Controlled via GameEngine.java. Each loop tracks 6 combat encounters (6 counters). The system allows a maximum of 2 consecutive runs (12 counters total). A 3rd attempt is strictly blocked by creators decision throwing an OverLoopException, and requires a manual restart.
User Input, States & Files	Console-driven system using numbers stated in menus. Tracks runtime game states (deaths, wins, defeats, HPs). Handles file operations, state and input anomalies by throwing controlled exceptions, such as catching IOException via FileWriter inside the save() method.

V. CONCLUSION

Soul Reaper: The Hero's Last Stand demonstrates a complete object-oriented console game with a meaningful class hierarchy, polymorphic item and enemy systems, custom exception handling, controlled user input, try-catches, variety of methods and saved game summaries. The most important strength of the project is that the OOP concepts are prioritized and highly used. These concepts directly support the game's mechanics, including combat, inventory, shop behavior, map exploration, and final-boss support events. With all the requirements meeting with the criterias, we as the creators of this game wish you would enjoy it.